

Tokio + IDX: NUMA0 per-message, per-thread characteristics

Use NUMA-constrained data. For each message size, sweep submitter thread count and compare Tokio to raw hardware at matched NUMA0 points.

Device/WQ

/dev/dsa/wq0.1 shared WQ

Operation

direct/no-batch DSA memmove;
batch_n=1

Placement

NUMA node 0 CPU + memory

Threads

1, 2, 4, 8, 16, 32, 64

128 B, 32T

1.18x

46.4 / 39.4 Mops/s

256 B, 4T

1.08x

11.4 / 10.6 Mops/s

128 B, 16T

0.81x

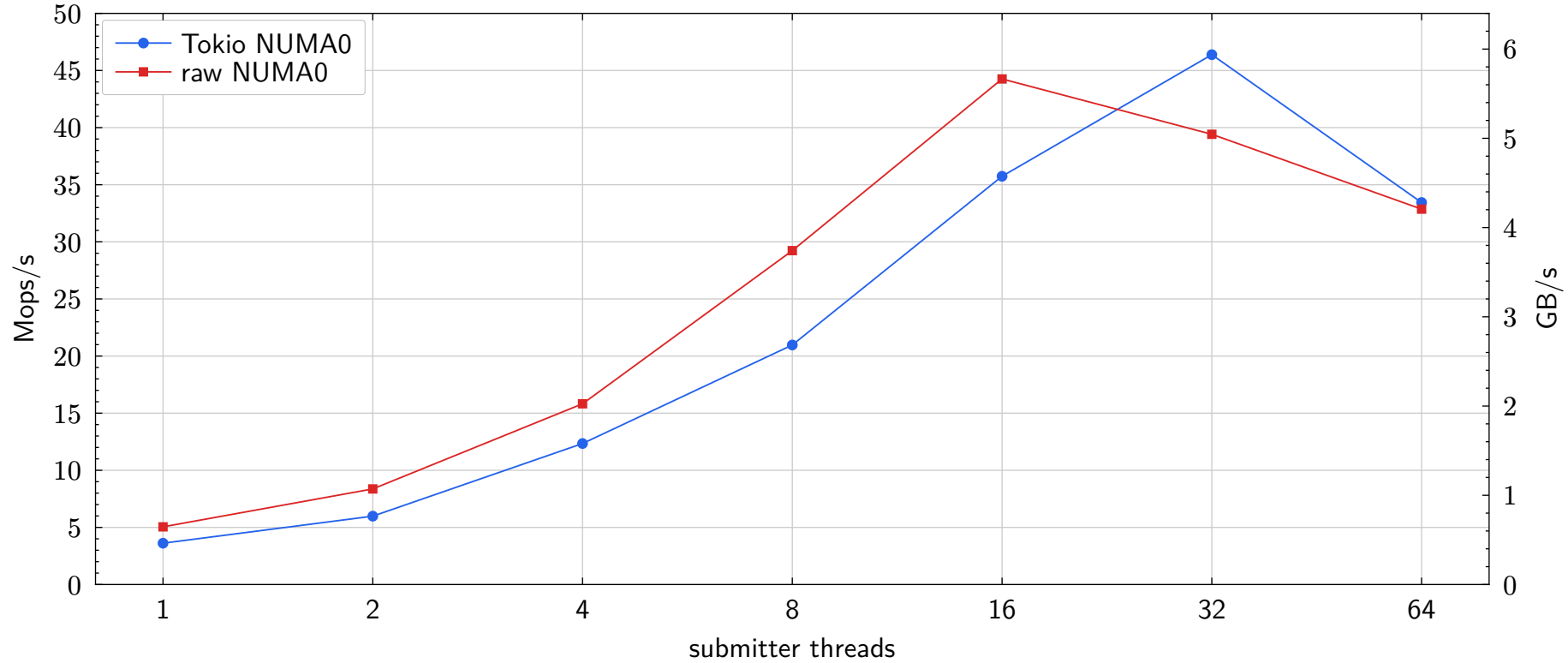
35.7 / 44.3 Mops/s

Plot rule

Generated from the scenarios data array. Each slide fixes one message size and c=128, then sweeps all matched NUMA0 Tokio and raw thread counts.

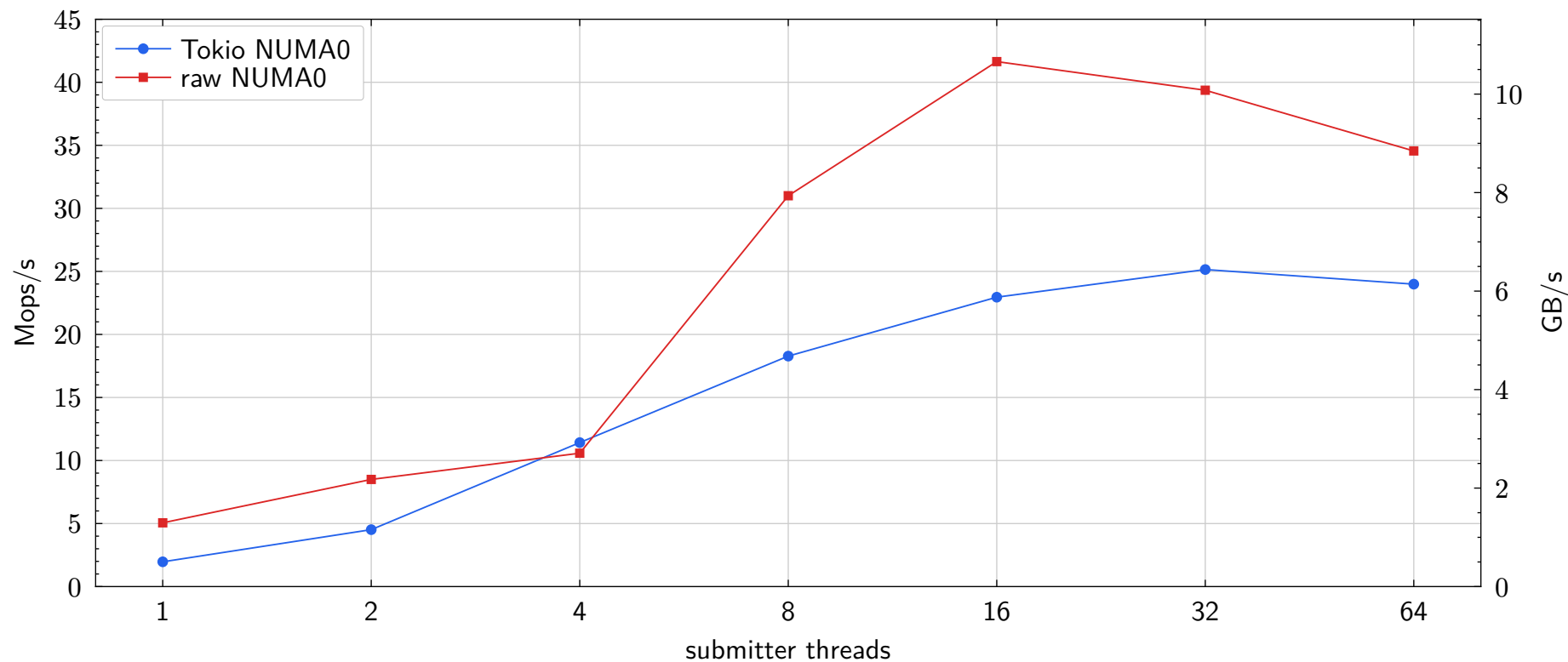
128 B: NUMA0 per-thread scaling at c=128

NUMA-constrained Tokio keeps scaling through 32 threads; the refreshed raw sweep now covers the same thread counts.



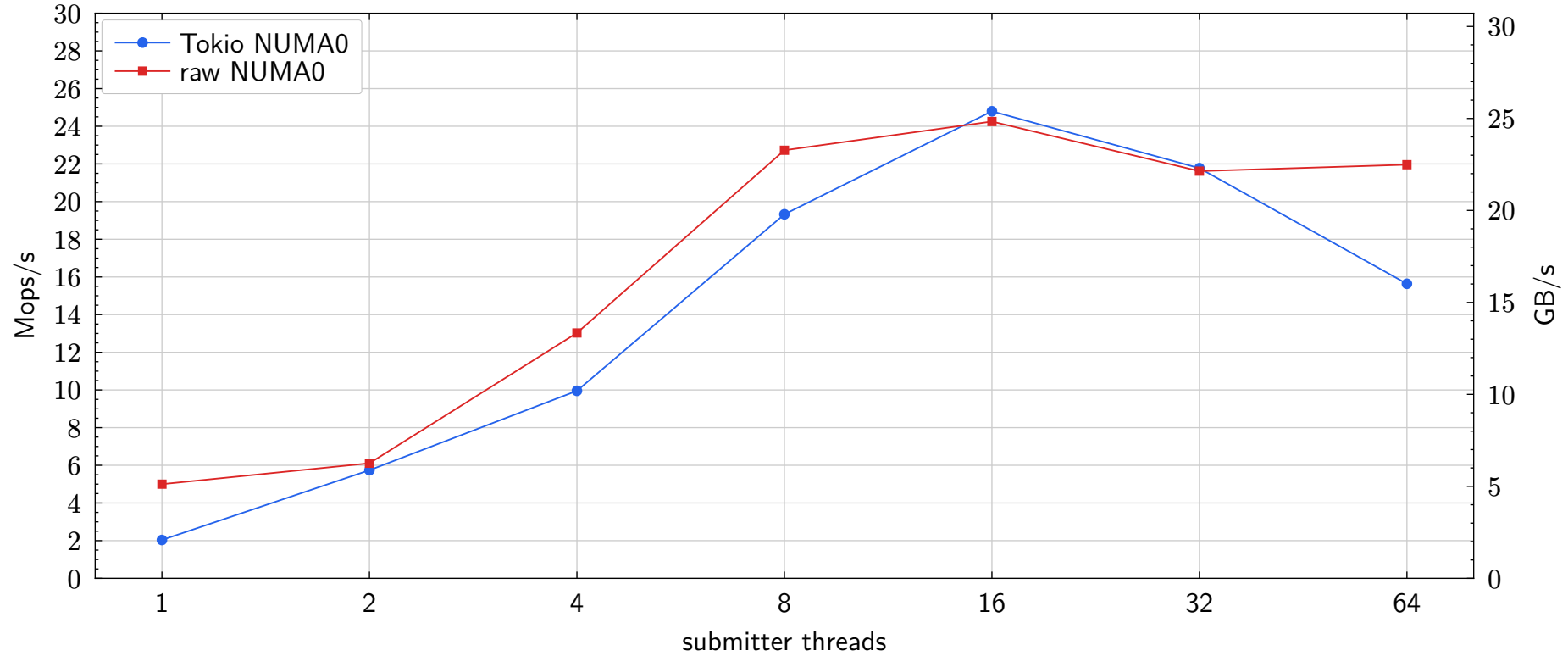
256 B: NUMA0 per-thread scaling at c=128

Raw is generally higher for 256 B; Tokio's best matched point is around 4 threads.



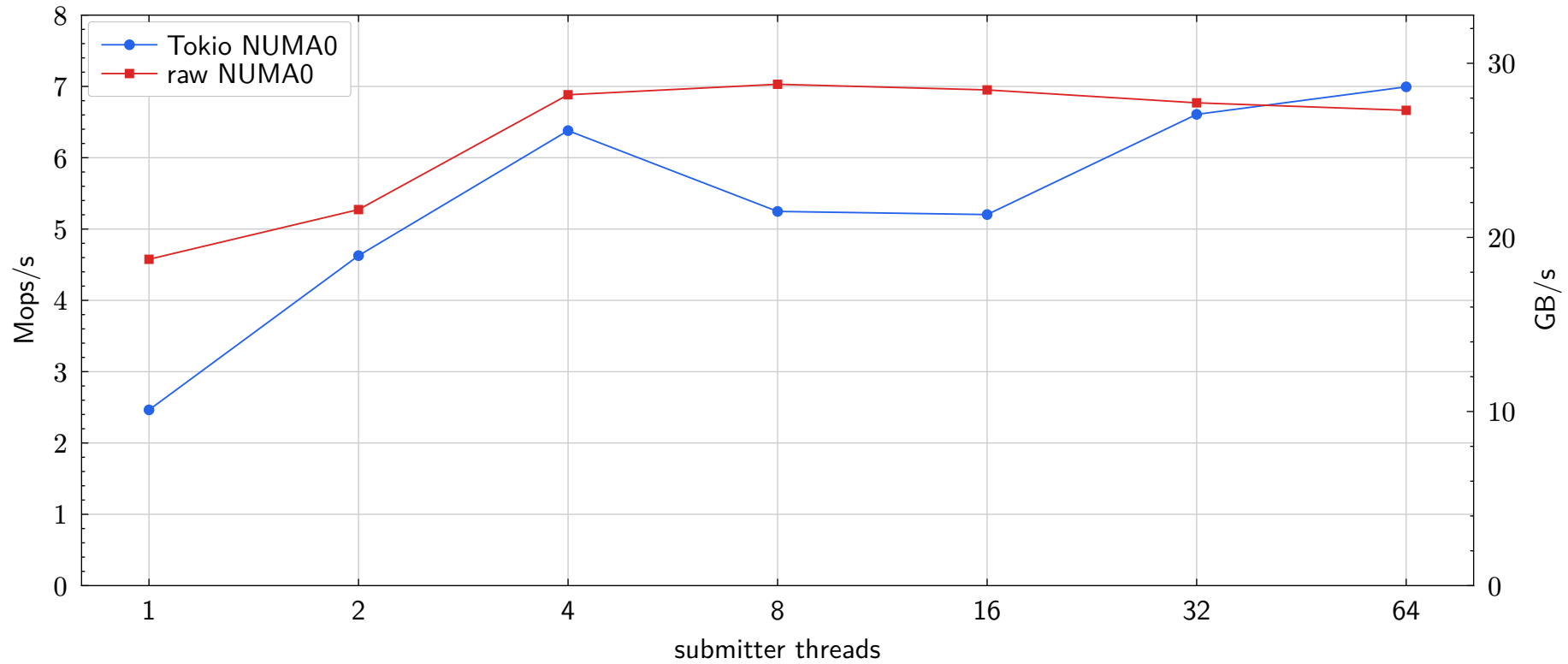
1 KiB: NUMA0 per-thread scaling at c=128

Tokio is close at 16–32 threads; raw is steadier at high thread counts.



4 KiB: NUMA0 per-thread scaling at c=128

Both paths reach high byte throughput; Tokio is closest at 32–64 threads.



Sources: NUMA0 Tokio JSONL files and refreshed NUMA0 raw hw-eval JSON files under shared_thread_sweep_numa0/. Left axis: Mops/s. Right axis: GB/s.